Application Debugging



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Good practices



Good practices

- Some good practices can allow you to save time before even needing to use a debugger
- Compiler are now smart enough to detect a wide range of errors at compile-time using warnings
  - Using `-Werror -Wall -Wextra` is recommended if possible to catch errors as early as possible
- Compilers now offer static analysis capabilities
  - GCC allows to do so using the -fanalyzer flag
  - LLVM provides dedicated tools that can be used in build process
- You can also enable component-specific helpers/hardening
  - If you are using the GNU C library, you can for example enable *_FORTIFY_SOURCE* macro to add runtime checks on inputs (e.g: buffers)



Instrumenting code crashes



Instrumenting code crashes

- Call stacks can be manually requested/generated in applications
- This can be done either with specialized libraries (eg: libunwind) or even directly the C library, e.g by using `backtrace()` (man 3 backtrace) with glibc:

```
char **backtrace_symbols(void *const *buffer, int size);
```

- We can wire those custom backtraces on signals thanks to `sigaction()` (man 2 sigaction)
  - `SIGSEGV` signal can be hooked to dump a call stack of an invalid memory access

```
int sigaction(int sig, const struct sigaction *restrict act,
              struct sigaction *restrict oact);
```

- Some libraries like libsigsegv ease the instrumentation of those signals



Custom code crash report (1/2)

```
[...]
void callee(void *ptr) {
  int *myptr = (int *)ptr;
  printf("Executing suspicious operation\n");
  myptr[2] = 0;
}

void caller(void) {
  void *ptr = NULL;
  callee(ptr);
}

void segfault_handler(int sig) {
  void *array[20];
  size_t size;
  char msg[]= "Segmentation fault!\n";

  write(STDERR_FILENO, msg, sizeof(msg));
  size = backtrace(array, 20);
  backtrace_symbols_fd(array, size, STDERR_FILENO);
  exit(1);
}
```

```
int main() {
  const struct sigaction act = {
    .sa_handler = segfault_handler,
    .sa_mask = 0,
    .sa_flags = 0,
  };
  if (sigaction(SIGSEGV, &act, NULL))
    exit(EXIT_FAILURE);
  printf("Calling a faulty function\n");
  caller();
  return 0;
}
```



Custom code crash report (2/2)

```
[root@arch-bootlin-alexis custom_backtrace]# ./main
Calling a faulty function
Executing suspicious operation
Segmentation fault!
./main(segfault_handler+0x60)[0x55c6e4c1723c]
/usr/lib/libc.so.6(+0x38f50)[0x7fecb0a95f50]
./main(callee+0x2b)[0x55c6e4c171b4]
./main(caller+0x1c)[0x55c6e4c171d9]
./main(main+0x2c)[0x55c6e4c1729a]
/usr/lib/libc.so.6(+0x23790)[0x7fecb0a80790]
/usr/lib/libc.so.6(__libc_start_main+0x8a)[0x7fecb0a8084a]
./main(_start+0x25)[0x55c6e4c170b5]
```

- Writing robust signal handlers is not easy
  - When your application has received a `SIGSEGV` signal, its execution can not really be trusted anymore
  - We are for example supposed to use only reentrant functions in signal handlers
  - Not following this rule may lead to undefined behavior



Building with debug information



Debugging with ELF files

- GDB uses ELF files since they contain the debugging information
- Debugging information uses the DWARF format
- Allows the debugger to match addresses and symbol names, call sites, etc
- Debugging information is generated by the compiler and included in the ELF file when compiled with `-g`
  - `-g1`: minimal debug information (enough for backtraces)
  - `-g2`: default debug level when using `-g`
  - `-g3`: includes extra debugging information (macro definitions)
- See GCC documentation about debugging for more information





Debugging with compiler optimizations

- Compiler optimizations (`-O<level>`) can lead to optimizing out some variables and function calls.
- Trying to display them with GDB will display
  - `$1 = <value optimized out>`
- If one wants to inspect variables and functions, it is possible to compile the code using `-O0` (no optimization).
  - *Note: The kernel can only be compiled with `-O2` or `-Os`*
- It is also possible to annotate function with compiler attributes:
  - `__attribute__((optimize("O0")))`
- Remove function `static` qualifier to avoid inlining the function
  - *Note: LTO (Link Time Optimization) can defeat this.*
- Set a specific variable as `volatile` to prevent the compiler from optimizing it out.
- having frame pointers enabled (`-fno-omit-frame-pointer`) may help GDB reconstruct stack frames
- having debug info for your main binary dependencies as well allows you to navigate in them



The ptrace system call



ptrace

- The *ptrace* mechanism allows processes to trace other processes by accessing tracee memory and register contents
- A tracer can observe and control the execution state of another process
- Works by attaching to a tracee process using the `ptrace()` system call (see man 2 ptrace)
- Can be executed directly using the `ptrace()` call but often used indirectly using other tools.

```
long ptrace(enum __ptrace_request request, pid_t pid, void *addr, void *data);
```

- Used by *GDB*, *strace* and all debugging tools that need access to the tracee process state

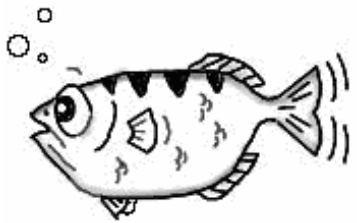


GDB



GDB: GNU Project Debugger

- The debugger on GNU/Linux, available for most embedded architectures.
- Supported languages: C, C++, Pascal, Objective-C, Fortran, Ada...
- Command-line interface
- Integration in many graphical IDEs
- Can be used to
  - control the execution of a running program, set breakpoints or change internal variables
  - to see what a program was doing when it crashed: post mortem analysis
- https://www.gnu.org/software/gdb/
- https://en.wikipedia.org/wiki/Gdb
- New alternative: *lldb* (https://lldb.llvm.org/) from the LLVM project.





GDB crash course (1/3)

- GDB is used mainly to debug a process by starting it with *gdb*
  - `$ gdb <program>`
- GDB can also be attached to running processes using the program PID
  - `$ gdb -p <pid>`
- When using GDB to start a program, the program needs to be run with
  - `(gdb) run [prog_arg1 [prog_arg2] ...]`



GDB crash course (2/3)

A few useful GDB commands

- `break foobar` (`b`)
  Put a breakpoint at the entry of function `foobar()`
- `break foobar.c:42`
  Put a breakpoint in `foobar.c`, line 42
- `print var`, `print $reg` or `print task->files[0].fd` (`p`)
  Print the variable `var`, the register `$reg` or a more complicated reference. GDB can also nicely display structures with all their members
- `info registers`
  Display architecture registers



GDB crash course (3/3)

- `continue` (`c`)
  Continue the execution after a breakpoint
- `next` (`n`)
  Continue to the next line, stepping over function calls
- `step` (`s`)
  Continue to the next line, entering into subfunctions
- `stepi` (`si`)
  Continue to the next instruction
- `finish`
  Execute up to function return



GDB advanced commands (1/3)

- `backtrace` (`bt`)
  Display the program stack
- `frame <n>` (`f <n>`)
  Select a specific frame from the backtrace, the number being the one displayed when using `backtrace` at the beginning of each line
- `info threads` (`i threads`)
  Display the list of threads that are available
- `thread <n>` (`t <n>`)
  Select thread number
- `info breakpoints` (`i b`)
  Display the list of breakpoints/watchpoints
- `delete <n>` (`d <n>`)
  Delete breakpoint



GDB advanced commands (2/3)

- `watch <variable>` or `watch <address>`
  Add a watchpoint on a specific variable/address.
- `break foobar.c:42 if condition`
  Break only if the specified condition is true
- `watch <variable> if condition`
  Trigger the watchpoint only if the specified condition is true
- `display <expr>`
  Automatically prints expression each time program stops
- `x/<n><u> <address>`
  Display memory at the provided address. `n` is the amount of memory to display, `u` is the type of data to be displayed (`b/h/w/g`). Instructions can be displayed using the `i` type.



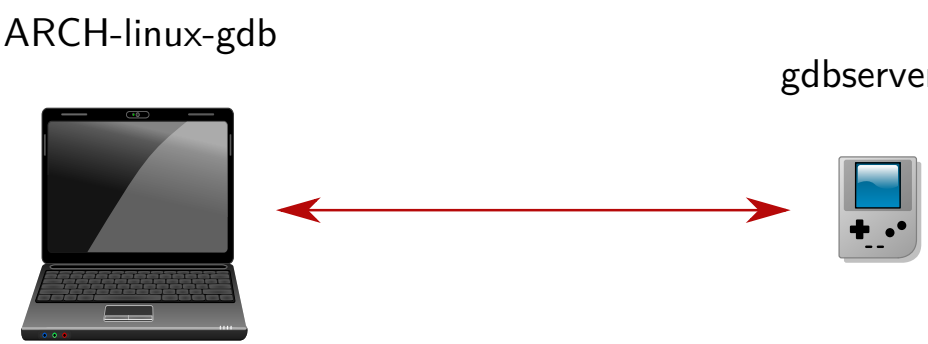
GDB advanced commands (3/3)

- `list <expr>`
  Display the source code associated to the current program counter location.
- `disassemble <location,start_offset,end_offset>` (`disas`)
  Display the assembly code that is currently executed.
- `print variable = value` (`p variable = value`)
  Modify the content of the specified variable with a new value
- `p function(arguments)`
  Execute a function using GDB. NOTE: be careful of any side effects that may happen when executing the function
- `p $newvar = value`
  Declare a new gdb variable that can be used locally or in command sequence
- `define <command_name>`
  Define a new command sequence. GDB will prompt for the sequence of commands.



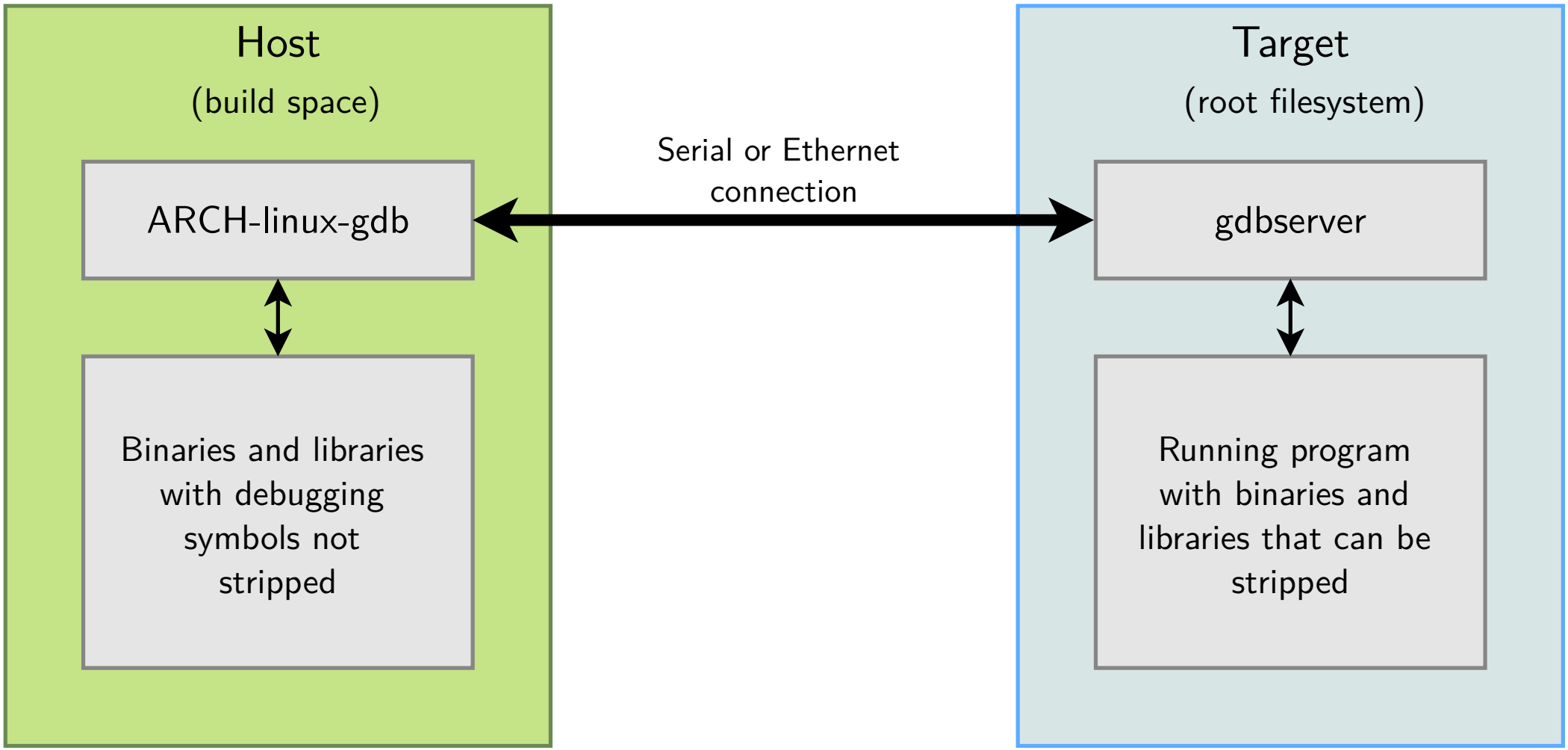
Remote debugging

- In a non-embedded environment, debugging takes place using `gdb` or one of its front-ends.
- `gdb` has direct access to the binary and libraries compiled with debugging symbols, which is often false for embedded systems (binaries are stripped, without debug_info) to save storage space.
- For the same reason, embedding the `gdb` program on embedded targets is rarely desirable (2.4 MB on x86).
- Remote debugging is preferred
  - `ARCH-linux-gdb` is used on the development workstation, offering all its features.
  - `gdbserver` is used on the target system (only 400 KB on arm).





Remote debugging: architecture





Remote debugging: target setup

- On the target, run a program through `gdbserver`.
  Program execution will not start immediately.
  `gdbserver :<port> <executable> <args>`
  `gdbserver /dev/ttyS0 <executable> <args>`
- Otherwise, attach `gdbserver` to an already running program:
  `gdbserver --attach :<port> <pid>`
- You can also start gdbserver without passing any program to start or attach (and set the target program later, on client side):
  `gdbserver --multi :<port>`



Remote debugging: host setup

- Then, on the host, start `ARCH-linux-gdb <executable>`, and use the following `gdb` commands:
  - To tell `gdb` where shared libraries are:
    `gdb> set sysroot <library-path>` (typically path to build space without `lib/`)
  - To connect to the target:
    `gdb> target remote <ip-addr>:<port>` (networking)
    `gdb> target remote /dev/ttyUSB0` (serial link)
    - Make sure to replace `target remote` with `target extended-remote` if you have started gdbserver with the `--multi` option
  - If you did not set the program to debug on gdbserver commandline:
    `gdb> set remote exec-file <path_to_program_on_target>`



Coredumps for post mortem analysis

- It is sometimes not possible to have a debugger attached when a crash occurs
- Fortunately, Linux can generate a `core` file (a snapshot of the whole process memory at the moment of the crash), in the ELF format. gdb can use this `core` file to let us analyze the state of the crashed application
- On the target
  - Use `ulimit -c unlimited` in the shell starting the application, to enable the generation of a `core` file when a crash occurs
  - The output name and path for the coredump file can be modified using `/proc/sys/kernel/core_pattern` (see man 5 core)
    - Example: `echo /tmp/mycore > /proc/sys/kernel/core_pattern`
  - Depending on the system configuration, the `core_pattern` file may be rewritten automatically by some software to handle core files or even disable core generation (eg: systemd)
- On the host
  - After the crash, transfer the `core` file from the target to the host, and run `ARCH-linux-gdb application-binary core-file`



minicoredumper

- Coredumps can be huge for complex applications
- minicoredumper is a userspace tool based on the standard core dump feature
  - Based on the possibility to redirect the core dump output to a user space program via a pipe
- Based on a JSON configuration file, it can:
  - save only the relevant sections (stack, heap, selected ELF sections)
  - compress the output file
  - save additional information from `/proc`
- https://github.com/diamon/minicoredumper
- "Efficient and Practical Capturing of Crash Data on Embedded Systems"
  - Presentation by minicoredumper author John Ogness
  - Video: https://www.youtube.com/watch?v=q2zmwrgLJGs
  - Slides: elinux.org/images/8/81/Eoss2023_ogness_minicoredumper.pdf

GDB: going further

- Tutorial: Debugging Embedded Devices using GDB - Chris Simmonds, 2020
  - Slides: https://elinux.org/images/0/01/Debugging-with-gdb-csimmonds-elce-2020.pdf
  - Video: https://www.youtube.com/watch?v=JGhAgd2a_Ck





GDB Python Extension

- GDB features a python integration, allowing to script some debugging operations
- When executing python under GDB, a module named *gdb* is available and all the GDB specific classes are accessible under this module
- Allows to add new types of commands, breakpoint, printers
  - Used by the kernel to create new commands with the python GDB scripts
- Allows full control and observability over the debugged program using GDB capabilities from Python scripts
  - Controlling execution, adding breakpoints, watchpoints, etc
  - Accessing the process memory, frames, symbols, etc





GDB Python Extension (1/2)

```
class PrintOpenFD(gdb.FinishBreakpoint):
  def __init__(self, file):
    self.file = file
    super(PrintOpenFD, self).__init__()

  def stop (self):
    print ("---> File " + self.file + " opened with fd " + str(self.return_value))
    return False

class PrintOpen(gdb.Breakpoint):
  def stop(self):
    PrintOpenFD(gdb.parse_and_eval("file").string())
    return False

class TraceFDs (gdb.Command):
  def __init__(self):
    super(TraceFDs, self).__init__("tracefds", gdb.COMMAND_USER)

  def invoke(self, arg, from_tty):
    print("Hooking open() with custom breakpoint")
    PrintOpen("open")

TraceFDs()
```

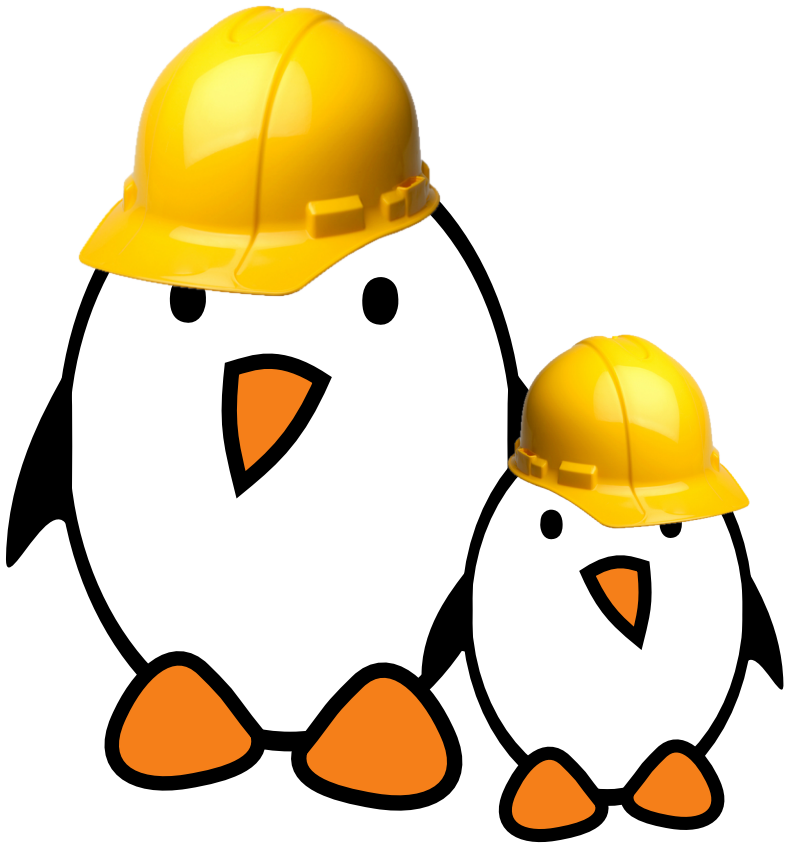


GDB Python Extension (2/2)

- Python scripts can be loaded using gdb `source` command
  - Or the script can be named `<program>-gdb.py` and will be loaded automatically by GDB

```
(gdb) source trace_fds.py
(gdb) tracefds
Hooking open() with custom breakpoint
Breakpoint 1 at 0x33e0
(gdb) run
Starting program: /usr/bin/touch foo bar
Temporary breakpoint 2 at 0x5555555587da
---> File foo opened with fd 3
Temporary breakpoint 3 at 0x5555555587da
---> File bar opened with fd 0
```

Practical lab - Solving an application crash



Debugging an application crash

- Code generation analysis with compiler-explorer
- Using GDB and its Python support
- Analyzing and using a coredump